

Filter FabJS

Programming Guide

Formula language reference, renderer compatibility, and practical filter tutorials

For Filter FabJS v2.4.7

August 2026

Inspired by the structure of the 1997 Filter Factory Programming Guide, but rewritten for Filter FabJS's browser-native formula engine.

Contents

- 0. About this guide
- 1. What is Filter FabJS?
- 2. Programming the Formula Language
 - 2.1 Pixel and channel model
 - 2.2 Native float math vs. legacy integer math
 - 2.3 Formula syntax and parser limits
 - 2.4 Constants and variables
 - 2.5 Operators and precedence
 - 2.6 Controls and mappings
 - 2.7 Image sampling
 - 2.8 Numeric and shaping functions
 - 2.9 Polar coordinates
 - 2.10 Procedural noise
 - 2.11 Gradients and patterns
 - 2.12 Analytic shapes and masks
 - 2.13 Blend modes
 - 2.14 Legacy/stateful functions
 - 2.15 CPU and WebGPU compatibility
- 3. Tutorials
- 4. Debugging, safety, and performance
- 5. Import/export notes
- 6. Quick reference
- Appendix A. Formula-writing checklist
- Appendix B. Corrections from classic Filter Factory assumptions

0. About this guide

This guide documents the formula language implemented by Filter FabJS v2.4.7. It is a programming reference and tutorial rather than an architectural specification. Its goal is to help authors write filters directly in the four RGBA formula fields, understand each expression, and predict when a formula can run on WebGPU or must fall back to the CPU Worker.

The supplied 1997 Filter Factory Programming Guide is used as a structural reference: fundamentals first, then variables, operators, and functions, followed by progressively richer image-processing tutorials. Filter FabJS is not a drop-in clone of the original language. It preserves familiar ideas while adding native floating-point math, bilinear sampling, deterministic procedural noise, gradients, patterns, analytic masks, blend helpers, renderer selection, and strict parser limits.

Every formula example in this guide follows the current Filter FabJS grammar. Examples intentionally avoid unsupported statement syntax such as assignments, semicolons, loops, and user-defined functions.

1. What is Filter FabJS?

Filter FabJS is a browser-native procedural RGBA image-filter editor. A filter is defined by four expressions: one each for red, green, blue, and alpha. Each expression is evaluated for every output pixel and can read the source pixel, sample other coordinates, use controls, generate procedural values, or combine those techniques.

The editor compiles parsed formulas into a typed intermediate representation (IR). In Auto mode, deterministic stateless formulas are sent to WebGPU when compatible. Unsupported, numerically unsafe for f32, or legacy formulas are evaluated by a CPU Worker.

1.1 The four-channel model

The four formula fields produce output channels R, G, B, and A. A formula may read any source channel regardless of its output field. The final result is clamped to 0..255 when written to the output image.

Pass-through filter

```
R: r
G: g
B: b
A: a
```

The current source pixel is always available through r, g, b, and a. The variable c means the current source channel: in the R field c is r, in the G field c is g, and so on.

1.2 Rendering model

- Auto: uses WebGPU when the parsed program is compatible and WebGPU is available; otherwise uses the CPU Worker.
- GPU: requests WebGPU, but still falls back to CPU for unsupported, unavailable, initialization-failed, or runtime-failed work.
- CPU: evaluates the typed IR in a Web Worker and supports the complete language, including legacy and stateful constructs.

WebGPU compatibility is decided from the typed IR, not from a preset flag. A small syntax or numeric change can move a formula from the GPU-compatible subset to CPU fallback.

2. Programming the Formula Language

2.1 Pixel and channel model

Coordinates use a conventional raster image system. x increases from left to right and y from top to bottom. The final valid integer pixel is (X-1,Y-1).

Symbol	Meaning	Typical range
r, g, b, a	Source red, green, blue, alpha	0..255
c	Source value for the current output channel	0..255
x, y	Current pixel coordinates	0..X-1, 0..Y-1
z / p	Current output-channel index	0..3
X / Y	Image width / height	positive
Z / P	Total channel count	4
i	Luminance from source RGB	approximately 0..255
u / v	Signed chroma in native float mode	about -55..55 / -78..78
d	Direction from image center in 1/1024 turns	about -512..512
m	Distance from image center	0..M
D	One complete angular turn	1024
M	Half the full image diagonal	$\sqrt{(X^2+Y^2)}/2$

2.2 Native float math vs. legacy integer math

Native Filter FabJS formulas use floating-point arithmetic, so fractional intermediate results are preserved.

Native float mode: approximately 170

$2/3 * 255$

Historic .afs imports use legacy integer compatibility mode. Values are evaluated with signed 32-bit integer behavior at expression and helper boundaries. Division truncates toward zero, multiplication uses signed 32-bit products, and overflow wraps as it did in Filter Factory-style arithmetic. Legacy `val()`, `map()`, `mix()`, `scl()`, and convolution helpers preserve their integer staging.

Legacy `sqr()` and `sqrt()` both use the Filter Factory integer square-root routine. Legacy `pow()` converts its arguments to signed 32-bit integers and applies compatibility rounding to the result. These rules are intentionally separate from native float semantics.

Legacy integer mode: 0, because 2/3 truncates to 0

$2/3 * 255$

Do not write new native filters around implicit integer truncation. Use `floor()`, `ceil()`, `round()`, or an explicit quantization expression when that behavior is part of the intended effect.

2.3 Formula syntax and parser limits

Each channel contains one expression. The language is expression-oriented, not JavaScript. It cannot declare variables, assign values, write loops, call browser APIs, or use semicolon-separated statements.

- Whitespace is ignored and identifiers are case-sensitive.
- Line comments beginning with `//` are allowed.
- Decimal and hexadecimal numeric literals are accepted.
- Scientific notation such as `1e-3` is not accepted; write `0.001` instead.
- Nested calls, arithmetic, comparisons, logical operators, and ternary selection are supported; comma sequencing and stateful functions are CPU-only.

Limit	Maximum
Formula text	8,192 characters
Tokens	4,096
Syntax nodes	4,096
Nesting depth	128

```
clamp(c+20,0,255) // brighten the current channel
```

These limits are deliberate safety boundaries. Imported filters are subject to the same parser budgets.

2.4 Constants and variables

2.4.1 Numeric constants

```
42
-12.5
0.25
0xFF
```

Native formulas use finite JavaScript-style numeric values. Hexadecimal literals are useful for integer constants, but color-channel arithmetic is usually clearer in decimal.

2.4.2 Primary variables

Variable	Meaning
r, g, b, a	Source RGBA channels at the current pixel.
c	Current source channel selected by the output field.
i	Luminance: $0.299R + 0.587G + 0.114B$.
u / v	Signed YUV-like chroma; native nominal bounds are -55..55 and -78..78.
x, y	Current pixel coordinates.
z, p	Current channel index: 0=R, 1=G, 2=B, 3=A.
d	Polar direction from image center; 1024 units is one turn.
m	Polar radius from image center in pixels.

2.4.3 Range and compatibility constants

Variable(s)	Value in native mode
R,G,B,A,C,I	255
U / V	110 / 156 native chroma spans
rmax..imax	255
umin / umax	-55 / 55
vmin / vmax	-78 / 78
dmin / dmax	-512 / 512
xmax / ymax	X / Y
mmax	M
rmin..zmin	0 except umin, vmin, dmin
t / tmin / tmax / total	0 / 0 / 1 / 1

Legacy .afs imports switch the chroma model to historic 0..255 U/V bounds. The still-image application keeps t at 0 and total at 1; those names exist for compatibility rather than animation.

2.5 Operators and precedence

Group	Operators	Notes
Unary	+ - ! ~	~ is CPU-only.
Multiply	* / %	Division/modulo by zero return 0.
Add	+ -	
Shift	<< >>	CPU-only.
Compare	< <= > >=	Returns 0 or 1.
Equality	== !=	Returns 0 or 1.
Bitwise	& ^	CPU-only; one shared precedence level.
Logical	&&	Short-circuit; one shared precedence level.
Ternary	condition ? yes : no	Expression selection.
Comma	,	CPU-only sequencing; lowest precedence.

Precedence is similar to C but not identical. In particular, && and || share one precedence level, as do &, ^, and |. Parenthesize mixed logical or bitwise expressions.

Conditional inversion

```
i>128 ? 255-c : c
```

Logical and comparison operators return numeric 0 or 1. Any nonzero value is true in a condition.

2.6 Controls and mappings

2.6.1 ctl(index)

ctl(i) reads one of eight UI controls. Valid indices are 0 through 7 and the raw range is 0..255.

```
clamp(c+ctl(0)-128,0,255)
```

2.6.2 val(index, low, high)

val() maps a raw control continuously into a custom range. Native float mode preserves fractions; legacy mode applies its integer mapping stages.

```
val(0, -100, 100)
```

low may exceed high, which reverses the control direction.

2.6.3 map(index, value)

map(i,v) uses control pair i as a two-point remapping curve. The input is clamped to 0..255. Reversing the controls reverses the output ramp.

```
map(0,i)
```

For most new filters, val() is easier to reason about. map() is useful for adjustable low/high threshold pairs.

2.7 Image sampling

Function	Purpose
src(x,y,z)	Nearest-neighbor sample with edge clamping.
srcLinear(x,y,z)	Bilinear sample with edge clamping.
srcWrap(x,y,z)	Nearest sample with toroidal wraparound.
srcMirror(x,y,z)	Nearest sample with mirrored edge repetition.
rad(angle,distance,z)	Nearest polar sample around image center.

Sampling reads the immutable original source image, not partially rendered output. This prevents feedback loops and keeps deterministic formulas parallelizable.

Horizontal mirror

```
src(X-1-x,y,z)
```

Subpixel red-channel offset

```
srcLinear(x+val(0,-32,32),y+val(1,-32,32),0)
```

2.7.1 Edge behavior

`src()` and `srcLinear()` clamp to the nearest edge. `srcWrap()` wraps across opposite edges. `srcMirror()` reflects coordinates back into the image. Clamp smears at large offsets, wrap suits seamless displacement, and mirror often hides borders in blur-like transforms.

2.7.2 Legacy aliases

`src0()`, `src1()`, `rad0()`, `rad1()`, `cnv0()`, `cnv1()`, and `r0/r1`-style variables are compatibility aliases to the same single source image. Filter FabJS does not expose two simultaneous image sources through these names.

2.8 Numeric and shaping functions

Function	Meaning	WebGPU
min / max / abs	Minimum, maximum, and absolute value.	Yes
clamp / floor / ceil / round	Range clamp and explicit quantization.	Yes
fract / sign	Fractional part and sign.	Yes
sqr(x)	Native float: $x*x$. Legacy: Filter Factory integer square root.	Native only
sqrt(x)	Native float: $\sqrt{\max(0,x)}$. Legacy: integer square-root alias.	Native only
pow(x,y)	Power; legacy mode uses integer arguments and compatibility rounding.	No
add / sub / dif	Classic bounded-difference helpers; integer-staged in legacy mode.	Yes in native
scl / mix	Range mapping and weighted mixing; integer-staged in legacy mode.	Yes in native
lerp	Interpolation; t may be 0..1 or 0..255.	Yes
step / smoothstep	Hard and smooth threshold transitions.	Yes
bias / gain	Curve shaping for normalized values.	Yes

The native and legacy meanings of `sqr()` are intentionally different. Native filters use `sqr()` for squaring and `sqrt()` for square root. Historic AFS/legacy programs preserve Filter Factory's integer square-root meaning for both names.

Contrast

```
clamp((c-128)*val(1,0.25,2.5)+128,0,255)
```

`bias()` and `gain()` clamp their first argument to 0..1. A second parameter above 1 is interpreted on the 255 scale.

2.9 Polar coordinates

Filter FabJS uses 1024 angular units per full turn. Zero points right, 256 down, 512 left, and -256 up, matching screen coordinates where y increases downward.

Function / variable	Meaning
d	Angle of current pixel from image center.
m	Distance from image center.
c2d(dx,dy)	Cartesian offset to angle units.
c2m(dx,dy)	Cartesian offset to radius.
r2x / r2y	Polar coordinate offsets.
rad(angle,radius,z)	Sample around image center.

Function / variable	Meaning
sin / cos	Amplitude scaled to about +/-512.
tan	1024*tan(angle); grows rapidly near quarter turns.

Polar twist/radial offset

```
rad(d+val(0, -64, 64), m+val(1, -40, 40), z)
```

Divide sin() or cos() by 512 when normal -1..1 trigonometric output is required.

2.10 Procedural noise

Function	Return / use
hash2(x,y,seed)	Deterministic 0..1 hash.
valueNoise(x,y,scale,seed)	Smooth value noise, 0..1.
perlin(x,y,scale,seed)	Gradient noise normalized to 0..1.
worleyF1 / worleyF2	Nearest and second-nearest feature distance.
fbm(...)	Multi-octave Perlin fractal sum.
turbulence(...)	Absolute-value multi-octave noise.
ridged(...)	Squared ridge-style multi-octave noise.
periodicNoise(...)	Tileable 0..1 value noise.

Noise scale is measured in pixels. FBM octaves are clamped to 1..12, lacunarity stays above 1, and gain is clamped to 0.01..0.99.

WebGPU uses f32 values. If an integer noise argument is not exactly representable as f32 - for example seed 16777217 - compatibility analysis selects the CPU Worker so the exact integer is preserved. Adjacent representable integers, such as 16777216, may remain GPU-compatible.

Fractal clouds

```
fbm(x,y,val(0,12,180),5,2,0.5,val(1,1,9999))*255
```

Cellular edge field

```
(worleyF2(x,y,40,17)-worleyF1(x,y,40,17))*900
```

2.10.1 Stateful random: rnd() and rst()

rnd(a,b) returns an inclusive pseudo-random integer. rst(seed) resets the sequential generator and returns 0. Because their result depends on evaluation order, both are CPU-only. Prefer hash2() and stateless noise for new filters.

2.11 Gradients and patterns

Function	Meaning
wrap / mirror	Positive modulo and ping-pong coordinate reflection.
linearGrad	0..1 projection along a line segment.
radialGrad	1 at center, falling to 0 at radius.
angularGrad	Repeating 0..1 angle around center.
checker	Alternating 0/1 cells.
brick	Brick-interior mask.
grid	Anti-aliased repeating line mask.

```

checker(x,y,val(0,4,64),val(1,4,64))*255
linearGrad(x,y,0,0,X-1,Y-1)*255
radialGrad(x,y,X/2,Y/2,min(X,Y)/2)*255

```


angularGrad() treats |offset|<=1 as a normalized turn and larger values as angle units. brick() similarly treats |offset|<=1 as a fraction of brick width and larger values as pixels.

2.12 Analytic shapes and masks

Analytic shapes return normalized masks: 0 outside and 1 inside, with a smooth transition where feathering is nonzero. Coordinates and dimensions are pixels; negative dimensions are treated by absolute value.

Function	Signature
line	line(x,y,ax,ay,bx,by,width,feather)
circle	circle(x,y,cx,cy,radius,feather)
ring	ring(x,y,cx,cy,radius,width,feather)
box	box(x,y,cx,cy,width,height,rotation,feather)
triangle	triangle(x,y,ax,ay,bx,by,cx,cy,feather)
grid	grid(x,y,cellWidth,cellHeight,lineWidth,feather)
sierpinski	sierpinski(x,y,cx,cy,size,depth,feather)

box() rotation uses 1024 units per turn. sierpinski() clamps depth to 0..10 and feathers both its outer edge and internal holes.

```
circle(x,y,X/2,Y/2,min(X,Y)*0.25,val(0,0,4))*255
```

2.12.1 Combining masks

Use max() for union, multiplication for intersection, and subtraction followed by clamp() for a cutout.

```
max(circle(x,y,X/2,Y/2,80,1),box(x,y,X/2,Y/2,120,60,128,1))*255
clamp(circle(x,y,X/2,Y/2,80,1)-circle(x,y,X/2,Y/2,60,1),0,1)*255
```

2.12.2 Sierpinski gasket

```
sierpinski(x,y,X/2,Y/2,min(X,Y)*0.9,val(0,2,9),val(1,0,2))*255
```

This is a true finite-depth gasket mask. Multiply it by channel-specific values or pass it directly to lerp() for compositing.

2.13 Blend modes

Function	Definition
multiply	$a*b/255$
screen	$255-(255-a)*(255-b)/255$
overlay	Multiply below midtone, screen-like above.
softLight	Soft-light curve.
difference	$\text{abs}(a-b)$

Each blend accepts optional opacity in 0..1 or 0..255. Inputs are clamped to 0..255, then the result is interpolated from the first input.

```
multiply(c,128,ctl(0))
```

2.14 Legacy and stateful functions

Feature	Meaning	Renderer
get(i) / put(v,i)	Read/write one of 256 shared cells.	CPU only
rnd(a,b) / rst(seed)	Sequential random and reset.	CPU only
pow(a,b)	Power function.	CPU only
~ & ^ << >>	Bitwise and shift operators.	CPU only
comma	Evaluate left, then right.	CPU only

The shared cells persist across pixels for one render and are cleared before the next. These features exist for historic compatibility, not as a foundation for new parallel filters.

2.15 CPU and WebGPU compatibility

Native float formulas are WebGPU-compatible when they use the stateless subset and every numeric literal can be represented safely by WGSL f32 semantics.

Language feature	WebGPU	CPU Worker
Arithmetic / comparisons / logical / ternary	Yes	Yes
Sampling / polar / 3x3 convolution	Yes	Yes
Noise / gradients / patterns / shapes	Yes*	Yes
Blend helpers	Yes	Yes
Legacy integer math	No	Yes
rnd/rst, get/put	No	Yes
Bitwise, shifts, comma	No	Yes
pow()	No	Yes
Literal outside f32 range or nonzero f32 underflow	No	Yes
Inexact integer noise argument	No	Yes

* Noise remains GPU-compatible only when integer arguments that require exactness are exactly representable as f32. Compatibility analysis sends overflow, underflow, and inexact-integer cases to CPU before shader creation. This is a correctness fallback, not a formula error.

In Auto mode, unsupported operations do not invalidate a filter. Small CPU/GPU floating-point differences remain possible around transcendental functions and threshold edges.

3. Tutorials

3.1 Pass through and invert

Pass through

```
R: r
G: g
B: b
A: a
```

Invert RGB, preserve alpha

```
R: 255-r
G: 255-g
B: 255-b
A: a
```

Preserving alpha with a is the safest default for effects that alter only color.

3.2 Luminance grayscale

```
R: i
G: i
B: i
A: a
```

i is computed from source RGB using weighted luminance.

3.3 Brightness and contrast

Brightness for R, G, and B

```
clamp(c+val(0,-100,100),0,255)
```

Contrast for R, G, and B

```
clamp((c-128)*val(1,0.25,2.5)+128,0,255)
```

Contrast expands or compresses distance from mid-gray. The final clamp keeps the display range valid.

3.4 Duotone

```
R: scl(i,0,255,ctl(0),ctl(3))
```

```
G: scl(i,0,255,ctl(1),ctl(4))
```

```
B: scl(i,0,255,ctl(2),ctl(5))
```

```
A: a
```

Controls 0-2 define the shadow color and controls 3-5 the highlight color.

3.5 Mirroring and offsets

Horizontal mirror

```
src(X-1-x,y,z)
```

Vertical mirror

```
src(x,Y-1-y,z)
```

Wrapped 2D offset

```
srcWrap(x+val(0,-X,X),y+val(1,-Y,Y),z)
```

The -1 in mirror formulas is essential because X and Y are dimensions, not final indices.

3.6 RGB channel split

```
R: srcLinear(x+val(0,-32,32),y+val(1,-32,32),0)
```

```
G: srcLinear(x+val(2,-32,32),y+val(3,-32,32),1)
```

```
B: srcLinear(x+val(4,-32,32),y+val(5,-32,32),2)
```

```
A: a
```

srcLinear() makes subpixel shifts smooth. Use srcWrap() when wraparound seams are preferable to edge clamping.

3.7 Mosaic

```
srcLinear(floor(x/val(0,2,64))*val(0,2,64)+val(0,2,64)/2,
```

```
floor(y/val(1,2,64))*val(1,2,64)+val(1,2,64)/2,z)
```

The formula finds the center of the current block and samples that location.

3.8 Procedural checker and gradients

```
checker(x,y,val(0,4,64),val(1,4,64))*255
```

```
linearGrad(x,y,0,0,X-1,Y-1)*255
```

```
radialGrad(x,y,X/2,Y/2,min(X,Y)/2)*255
```

Generators return normalized masks or fields. Multiply by 255 for a full grayscale image; keep 0..1 values for compositing.

3.9 Fractal clouds

```
fbm(x,y, val(0,12,180),5,2,0.5, val(1,1,9999))*255
```

Control 0 changes feature size and control 1 is a deterministic seed. Reduce octaves for lower cost.

3.10 Noise displacement

```
srcLinear(x+(valueNoise(x,y, val(0,8,120), val(2,1,9999))-0.5)*val(1,0,52),  
y+(valueNoise(x+431,y+719, val(0,8,120), val(2,1,9999))-0.5)*val(1,0,52), z)
```

Two decorrelated value-noise fields displace x and y. Subtracting 0.5 centers displacement around zero.

3.11 Analytic shape composition

```
max(circle(x,y,X/2,Y/2,80,1), box(x,y,X/2,Y/2,120,60,128,1))*255  
lerp(c,255, circle(x,y,X/2,Y/2,80,2))
```

Because lerp() accepts 0..1 masks directly, shape functions compose naturally with image channels.

3.12 Sierpinski fractal

```
R: lerp(r,238*sierpinski(x,y,X/2,Y/2,min(X,Y)*0.9, val(0,2,9), val(1,0,2)),ctl(7))  
G: lerp(g,232*sierpinski(x,y,X/2,Y/2,min(X,Y)*0.9, val(0,2,9), val(1,0,2)),ctl(7))  
B: lerp(b,214*sierpinski(x,y,X/2,Y/2,min(X,Y)*0.9, val(0,2,9), val(1,0,2)),ctl(7))  
A: a
```

The mask is GPU-compatible and finite-depth. Depth is capped at 10.

3.13 Convolution: blur and sharpen

3x3 box blur

```
cnv(1,1,1,1,1,1,1,1,9)
```

3x3 sharpen

```
cnv(0,-1,0,-1,5,-1,0,-1,0,1)
```

cnv() uses a row-major 3x3 kernel and a divisor. A zero divisor returns 0; edge sampling is clamped. Larger multi-pass operations require a true pass graph rather than deeply nested approximations.

3.14 Blend helpers

```
multiply(c,128,ctl(0))
```

Control 0 is opacity: 0 returns c and 255 returns the full multiply result.

3.15 Conditional threshold effects

```
i>128?255-c:c
```

Ternary expressions are GPU-compatible. For soft transitions, smoothstep() generally anti-aliases better than a hard comparison.

4. Debugging, safety, and performance

4.1 Common syntax errors

Problem	Bad	Correct
Function arity	<code>abs(r,g)</code>	<code>abs(r)</code>
Unknown helper	<code>lerp(a,b,c,d)</code>	<code>lerp(a,b,t)</code>
Missing ternary colon	<code>i>128?255</code>	<code>i>128?255:0</code>
Semicolon statement	<code>c+20;</code>	<code>c+20</code>
Assignment	<code>x=20</code>	Use a direct expression
Scientific notation	<code>1e-3</code>	<code>0.001</code>
Wrong case	<code>Min(r,g)</code>	<code>min(r,g)</code>

4.2 Division and modulo by zero

Division or modulo by zero returns 0. This avoids exceptions but can hide bad control ranges. Protect denominators when zero indicates an invalid parameter.

```
c/max(1, val(0,0,255))
```

4.3 Clamping and normalized fields

Image channels usually use 0..255. Masks, gradients, hash/noise, and shaping helpers often use 0..1. Mixing the scales is a common source of weak or blown-out effects.

- Multiply a 0..1 field by 255 only when converting it into a full channel image.
- Pass a 0..1 mask directly to `lerp()` unless 0..255 opacity is intentional.
- Clamp intermediate channel math when later operations assume 0..255.

4.4 CPU fallback

For an unexpected CPU fallback, first look for `rnd/rst`, `get/put`, `pow`, bitwise operators, shifts, comma sequencing, or legacy math. A native stateless formula also falls back when a literal is outside f32 range, a nonzero literal underflows to zero in f32, or an integer noise argument cannot be represented exactly as f32.

4.5 Cost model

- Nearest sampling is cheaper than bilinear sampling.
- A single `hash2()` is cheaper than multi-octave noise.
- FBM, turbulence, and ridged cost roughly in proportion to octave count.
- Worley noise checks neighboring feature cells and is heavier than hash/value noise.
- `cnv()` always performs a 3x3 neighborhood convolution.
- Analytic shapes are bounded, but deeply layered masks still add arithmetic cost.

Prefer a smaller number of well-chosen primitives over repeated duplicate subexpressions. The compiler lowers logical and comparison expressions in one traversal, but the formula language does not provide user-defined local variables.

5. Import/export notes

Filter FabJS has a native JSON filter format and can import historic Filter Factory .afs files. Native v2 export is normalized. Historic imports run in legacy integer mode and use the same formula length, token, node, and nesting limits as native formulas.

When translating an old Filter Factory formula manually, check these compatibility points:

- Native float formulas preserve fractions; legacy imports use signed 32-bit integer behavior.
- Native `sqr(x)` squares and native `sqrt(x)` calculates square root. In historic AFS/legacy mode both names use Filter Factory integer square root.
- Use stateless noise for new work rather than the original random lookup model.
- Source 0/1 aliases do not create two independent inputs.
- Filter FabJS always works with four RGBA channels internally.
- Preview-size workarounds are unnecessary; formulas use actual render dimensions.

6. Quick reference

Category	Functions
Sampling	<code>src</code> , <code>srcLinear</code> , <code>srcWrap</code> , <code>srcMirror</code> , <code>rad</code> , <code>cnv</code>
Controls	<code>ctl</code> , <code>val</code> , <code>map</code>
Basic math	<code>min</code> , <code>max</code> , <code>abs</code> , <code>add</code> , <code>sub</code> , <code>dif</code> , <code>clamp</code> , <code>floor</code> , <code>ceil</code> , <code>round</code> , <code>fract</code> , <code>sign</code>
Range/curves	<code>mix</code> , <code>scl</code> , <code>lerp</code> , <code>step</code> , <code>smoothstep</code> , <code>bias</code> , <code>gain</code>
Powers/trig	<code>sqr</code> , <code>sqrt</code> , <code>pow</code> , <code>sin</code> , <code>cos</code> , <code>tan</code>
Polar helpers	<code>r2x</code> , <code>r2y</code> , <code>c2d</code> , <code>c2m</code>
Noise	<code>hash2</code> , <code>valueNoise</code> , <code>perlin</code> , <code>worleyF1</code> , <code>worleyF2</code> , <code>fbm</code> , <code>turbulence</code> , <code>ridged</code> , <code>periodicNoise</code>
Patterns	<code>wrap</code> , <code>mirror</code> , <code>linearGrad</code> , <code>radialGrad</code> , <code>angularGrad</code> , <code>checker</code> , <code>brick</code>
Masks	<code>line</code> , <code>circle</code> , <code>ring</code> , <code>box</code> , <code>triangle</code> , <code>grid</code> , <code>sierpinski</code>
Blends	<code>multiply</code> , <code>screen</code> , <code>overlay</code> , <code>softLight</code> , <code>difference</code>
Legacy/stateful	<code>rnd</code> , <code>rst</code> , <code>get</code> , <code>put</code> , <code>src0</code> , <code>src1</code> , <code>rad0</code> , <code>rad1</code> , <code>cnv0</code> , <code>cnv1</code>

6.1 Function signatures

Signature	Purpose
<code>src(x,y,z)</code>	clamped nearest sample
<code>srcLinear(x,y,z)</code>	clamped bilinear sample
<code>srcWrap</code> / <code>srcMirror</code>	wrapped / mirrored nearest sample
<code>rad(angle,radius,z)</code>	polar sample
<code>cnv(k00,...,k22,div)</code>	3x3 convolution
<code>ctl(i)</code>	raw control 0..255
<code>val(i,a,b)</code>	map control into a..b
<code>map(i,v)</code>	control-pair remapping
<code>scl(v,a,b,c,d)</code>	map range a..b into c..d
<code>lerp(a,b,t)</code>	interpolate with 0..1 or 0..255 t
<code>hash2(x,y,seed)</code>	deterministic 0..1 hash
<code>fbm(...)</code>	fractal Perlin sum
<code>linearGrad</code> / <code>radialGrad</code> / <code>angularGrad</code>	normalized gradients
<code>checker</code> / <code>brick</code>	procedural pattern masks
<code>line</code> / <code>circle</code> / <code>ring</code> / <code>box</code> / <code>triangle</code> / <code>grid</code>	analytic masks
<code>sierpinski(x,y,cx,cy,size,depth,feather)</code>	finite-depth gasket mask
<code>multiply</code> / <code>screen</code> / <code>overlay</code> / <code>softLight</code> / <code>difference</code>	blend helpers with optional opacity

Appendix A. Formula-writing checklist

- Start with pass-through and change one channel at a time.
- Decide whether each value is a 0..255 channel or a normalized 0..1 field.
- Use `c` when one expression should apply independently to R, G, and B.
- Use explicit channel indices when channels need different sampling.
- Use `X-1` and `Y-1` for final valid integer coordinates.
- Prefer `val()` when a semantic control range is known.
- Prefer stateless hash/noise over `rnd()` for GPU-friendly randomness.
- Use `srcLinear()` for subpixel displacement and nearest samplers for crisp shifts.
- Parenthesize mixed logical or bitwise expressions.
- Preserve alpha with `a` unless the effect intentionally changes transparency.
- Check renderer status after adding legacy, stateful, or f32-sensitive values.
- Keep formulas within parser budgets and avoid repeated giant subexpressions.

Appendix B. Corrections from classic Filter Factory assumptions

Historic Filter Factory guidance should not be copied literally into native Filter FabJS formulas.

Classic assumption	Filter FabJS v2.4.7
All arithmetic is integer.	Native filters use floating point. Historic AFS/legacy imports use signed 32-bit compatibility behavior.
<code>sqr(x)</code> means square root.	That remains true in legacy mode. In native float mode <code>sqr(x)</code> means $x*x$ and <code>sqrt(x)</code> is square root.
Only nearest <code>src()</code> exists.	<code>srcLinear()</code> , <code>srcWrap()</code> , and <code>srcMirror()</code> are native additions.
Randomness is mainly <code>rnd()</code> .	Prefer deterministic hash/noise for GPU-compatible work.
Only old trig/polar helpers exist.	Modern gradients, patterns, masks, and noise are built in.
<code>get/put</code> are general optimization tools.	They are stateful CPU-only compatibility features.
<code>src0/src1</code> imply two inputs.	They alias the same single source image.
A filter should work around tiny previews.	Use <code>X/Y</code> for resolution-independent design against actual render dimensions.

The key distinction is math mode: native `sqr()` deliberately squares, while historic AFS/legacy `sqr()` and `sqrt()` preserve Filter Factory integer square-root behavior. Treating the two modes as interchangeable can produce severe visual errors.

Source basis

This guide was prepared from the Filter FabJS v2.4.7 repository implementation, including the formula parser, CPU evaluator, WGSL compiler, renderer manager, formula and authoring references, project-status documentation, built-in examples, and chroma contracts. The 1997 Filter Factory Programming Guide was used as a structural and historical reference, not copied as current syntax.